TikuOS Persistent FRAM Key-Value Store

Simple. Ubiquitous. Intelligence, Everywhere.

Ambuj Varshney
ambuj@tiku-os.org

March 11, 2026

Outline

1. Why Persistent Storage?
2. Architecture & Data Structures
3. Platform Layering
4. API Reference
5. Usage Examples
6. API Summary
7. Test Suite
8. Summary

The Problem: Managing Non-Volatile Data on MCUs

Common approaches (and their issues):

- Raw FRAM writes scattered across code
- No validation after reboot — FRAM garbage looks like real data
- No wear tracking — silent degradation over years
- Firmware updates lose calibration data
- No consistent API for persistent config

TikuOS persistent store:

- Registry of key → FRAM buffer mappings
- Magic number validates entries across reboots
- Write-count tracking for wear monitoring
- Re-registration preserves data across firmware updates
- All buffers statically declared — zero dynamic allocation

FRAM Is Not Free

FRAM retains data across reboots, but has finite write endurance (~10^{15} cycles). Untracked hot writes can degrade cells over the device lifetime.

Ideal Use Cases

Device configuration, sensor calibration, boot counters, error logs, operational state that must survive power loss.

Persistent Entry Structure

kernel/memory/tiku_mem.h

```
typedef struct {
    char       key[TIKU_PERSIST_MAX_KEY_LEN];
    uint8_t   *fram_ptr;
    tiku_mem_arch_size_t value_len;
    tiku_mem_arch_size_t capacity;
    uint32_t   write_count;
    uint32_t   magic;
    uint8_t    valid;
} tiku_persist_entry_t;
```

Field	Purpose
key	Short string identifier
fram_ptr	Caller-provided FRAM buffer
value_len	Current stored data length
capacity	Maximum buffer size
write_count	Writes for wear tracking
magic	0x544B5553 if valid
valid	Non-zero if entry is live

Key Design

The store **does not own** the FRAM buffers — the application provides statically-declared FRAM regions. The store only manages the registry of mappings and metadata.

Store Structure & Constants

Store control block

```
typedef struct {
    tiku_persist_entry_t
        entries[TIKU_PERSIST_MAX_ENTRIES];
    tiku_mem_arch_size_t count;
} tiku_persist_store_t;
```

Compile-time constants

```
/* Max entries (default 16) */
#define TIKU_PERSIST_MAX_ENTRIES  16

/* Max key length including \0 */
#define TIKU_PERSIST_MAX_KEY_LEN  8

/* Magic: "TKUS" in ASCII hex */
#define TIKU_PERSIST_MAGIC  0x544B5553U

/* Wear warning threshold */
#define TIKU_PERSIST_WEAR_THRESHOLD \
    1000000000UL
```

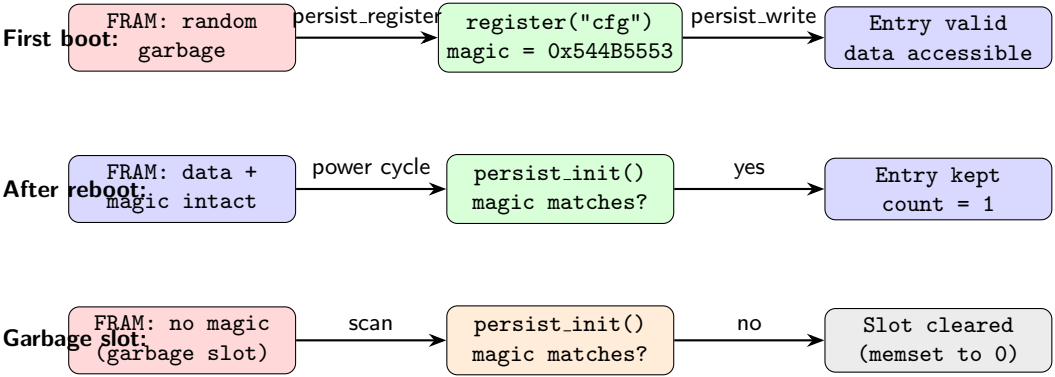
Extended error codes

```
typedef enum {
    TIKU_MEM_OK          =  0,
    TIKU_MEM_ERR_INVALID = -1,
    TIKU_MEM_ERR_NOMEM   = -2,
    TIKU_MEM_ERR_FULL    = -3,
    TIKU_MEM_ERR_NOT_FOUND = -4
} tiku_mem_err_t;
```

Configurability

All constants can be overridden with `#define` before including `tiku_mem.h`. A constrained device might use `MAX_ENTRIES=4` and `MAX_KEY_LEN=4`.

How Validation Works Across Reboots



HAL-Routed NVM Access

kernel/memory/tiku_persist.c (portable)

```
/* Read: delegates to arch layer */
tiku_mem_arch_nvm_read(
    buf, entry->fram_ptr,
    entry->value_len);

/* Write: delegates to arch layer */
tiku_mem_arch_nvm_write(
    entry->fram_ptr, data, data_len);
```

arch/msp430/tiku_mem_arch.c

```
/* FRAM is memory-mapped on MSP430,
   so NVM read is a byte-copy */
void tiku_mem_arch_nvm_read(
    uint8_t *dst,
    const uint8_t *src,
    tiku_mem_arch_size_t len)
{
    tiku_mem_arch_size_t i;
    for (i = 0; i < len; i++)
        dst[i] = src[i];
}

/* Caller must unlock MPU before
   calling nvm_write */
void tiku_mem_arch_nvm_write(
    uint8_t *dst,
    const uint8_t *src,
    tiku_mem_arch_size_t len)
{
    tiku_mem_arch_size_t i;
    for (i = 0; i < len; i++)
        dst[i] = src[i];
}
```

Why Not memcpy?

The kernel layer calls `tiku_mem_arch_nvm_read/write()` instead of `memcpy` directly. On MSP430, FRAM is memory-mapped so the arch layer uses byte-copy. On a platform with SPI Flash or EEPROM, the arch layer would use different access sequences.

Module File Organization

Layer	File	Role
Kernel (portable)	`kernel/memory/tiku_mem.h`	Types, constants, API declarations
	`kernel/memory/tiku_persist.c`	Store logic: find, init, register, R/W
	`kernel/memory/tiku_mpu.c`	MPU lock/unlock, scoped write
HAL	`hal/tiku_mem_hal.h`	Routes to arch; declares NVM functions
	`hal/tiku_mpu_hal.h`	Routes to arch; declares MPU functions
Arch (MSP430)	`arch/msp430/tiku_mem_arch.{h,c}`	NVM read/write (FRAM byte-copy)
	`arch/msp430/tiku_mpu_arch.c`	MPU register access
Tests	`tests/memory/test_mem_persist.c`	11 persist test cases
	`tests/memory/test_mem_common.c`	Host stubs + test main
	`tests/tiku_test_config.h`	Per-test enable flags

Porting to a New Platform

Only the arch layer changes:

1. Implement `tiku_mem_arch_nvm_read()` for your NVM technology

tiku_persist_init()

```c
tiku_mem_err_t tiku_persist_init(tiku_persist_store_t *store);
```

- Scans all entry slots — entries with correct `magic` and `valid` flag are kept
- All other slots are cleared with `memset` (FRAM garbage becomes clean)
- Sets `store->count` to the number of surviving entries
- Call **once at boot** before any other persist operation

Usage:

```c
/* Store in .persistent section so it
   resides in FRAM and survives reboots */
static tiku_persist_store_t
    __attribute__((section(".persistent")))
    store = {0};

void boot_init(void) {
    tiku_persist_init(&store);
    /* store.count tells you how many
       entries survived the reboot */
}
```

Why Scan for Magic?

tiku_persist_register()

```
tiku_mem_err_t tiku_persist_register(tiku_persist_store_t *store,
                                     const char *key,
                                     uint8_t *fram_buf,
                                     tiku_mem_arch_size_t capacity);
```

- If key already exists: updates FRAM pointer, **preserves existing data**
- If key is new: finds first empty slot, sets magic, clears write count
- Returns `TIKU_MEM_ERR_FULL` if all slots are occupied

Usage:

```
/* Declare FRAM buffer in .persistent section (GCC) */
static uint8_t
    __attribute__((section(".persistent")))
    cfg_fram[32] = {0};

tiku_mem_err_t err = tiku_persist_register(
    &store, "cfg", cfg_fram, sizeof(cfg_fram));
```

Firmware Update Survival

When the application is reflashed and re-registers the same key, the existing FRAM data and write count are preserved. Only the buffer pointer is updated (it may have moved in the new binary).

tiku_persist_read() & tiku_persist_write()

Read: FRAM → SRAM

```
tiku_mem_err_t tiku_persist_read(
    tiku_persist_store_t *store,
    const char *key,
    uint8_t *buf,
    tiku_mem_arch_size_t buf_size,
    tiku_mem_arch_size_t *out_len);
```

- Copies from FRAM into caller's SRAM buffer
- Reports required size via `out_len` even on failure
- Returns `ERR_NOMEM` if buffer too small
- Returns `ERR_NOT_FOUND` if key missing

Write: SRAM → FRAM

```
tiku_mem_err_t tiku_persist_write(
    tiku_persist_store_t *store,
    const char *key,
    const uint8_t *data,
    tiku_mem_arch_size_t data_len);
```

- Copies data into FRAM buffer
- Updates `value_len`
- Increments `write_count`
- Returns `ERR_NOMEM` if data exceeds capacity

Why Copy to SRAM?

FRAM has 1–2 wait states at higher clock speeds. Copying to SRAM gives faster subsequent access.

MPU Not Managed

The caller must unlock the MPU before calling `persist_write`. This lets the application batch multiple writes in one critical section.

tiku_persist_delete() & tiku_persist_wear_check()

Delete an entry

```
tiku_mem_err_t tiku_persist_delete(
    tiku_persist_store_t *store,
    const char *key);
```

- Clears the entry slot with `memset`
- Decrements `store->count`
- Key can no longer be found or read
- Slot becomes available for new registrations

Check wear level

```
int tiku_persist_wear_check(
    tiku_persist_store_t *store,
    const char *key,
    uint32_t *write_count);
```

- Returns 0 if within limits
- Returns 1 if `write_count` $\geq$ threshold
- Optionally outputs exact write count
- Threshold default: 10^9 (configurable)

Why Track Wear?

FRAM endurance is ~10^{15} cycles, but a counter written every second reaches 10^9 in ~31 years. Early warning lets the application relocate hot keys.

When to Use a Persistent Store

Persistent Store IS Right When...

- Device configuration must survive reboots
- Sensor calibration set once at factory
- Boot counter or error log for diagnostics
- Firmware updates must not lose user data
- You need wear tracking for hot FRAM cells

Persistent Store Is NOT Needed When...

- Data is recomputed from sensors every boot
- Values change every millisecond (use SRAM)
- You only need scratch memory (use arena/pool)
- Data is streamed out and never re-read locally

Rule of Thumb

If the answer to "does this data need to be the same after the next power cycle?" is yes, put it in the persistent store. If the answer is "no" or "it will be re-derived from hardware", keep it in SRAM and save the FRAM write cycles.

Example 1: Device Configuration with Defaults

```c
/* FRAM-backed buffers (survive reboots) */
static uint8_t __attribute__((section(".persistent")))
    cfg_fram[32] = {0};
static tiku_persist_store_t __attribute__((section(".persistent")))
    store = {0};

void app_init(void) {
    tiku_persist_init(&store);     /* recover valid entries from previous boot */
    tiku_persist_register(&store, "cfg", cfg_fram, sizeof(cfg_fram));

    /* Try to load saved config into fast SRAM copy */
    uint8_t config[32];
    tiku_mem_arch_size_t len;

    if (tiku_persist_read(&store, "cfg", config, sizeof(config), &len)
        == TIKU_MEM_OK) {
        apply_config(config, len);            /* use saved config */
    } else {
        load_defaults(config, &len);          /* first boot: apply defaults */
        tiku_persist_write(&store, "cfg", config, len);  /* save for next boot */
    }
}
```

Pattern: Read-or-Default

Example 2: Boot Counter & Wear Monitoring

```c
static uint8_t __attribute__((section(".persistent")))
    counter_fram[4] = {0};

void track_boot_count(void) {
    uint32_t boot_count = 0;
    tiku_mem_arch_size_t len;

    tiku_persist_register(&store, "boots", counter_fram,
                          sizeof(counter_fram));

    /* Read existing count (survives reboots) */
    if (tiku_persist_read(&store, "boots", (uint8_t *)&boot_count,
                          sizeof(boot_count), &len) != TIKU_MEM_OK) {
        boot_count = 0;  /* first boot */
    }

    boot_count++;
    tiku_persist_write(&store, "boots", (uint8_t *)&boot_count,
                       sizeof(boot_count));

    /* Check FRAM wear on hot keys */
    uint32_t writes;
    if (tiku_persist_wear_check(&store, "boots", &writes) == 1) {
        uart_puts("WARN: boot counter approaching FRAM wear limit\r\n");
    }
}
```

Example 3: Firmware Update Survival

Firmware v1.0

```c
/* FRAM buffer at fixed address */
static uint8_t
  __attribute__((section(".persistent")))
  cal_fram[16] = {0};

void v1_init(void) {
    tiku_persist_init(&store);
    tiku_persist_register(&store,
        "cal", cal_fram, 16);

    /* Write factory calibration */
    uint8_t cal[] = {0x42, 0x37};
    tiku_persist_write(&store,
        "cal", cal, 2);
}
/* Device ships with "cal" = {0x42,0x37}
   write_count = 1 */
```

Firmware v2.0 (reflashed)

```c
/* Buffer may be at a different address
   in the new binary */
static uint8_t
  __attribute__((section(".persistent")))
  cal_fram[16] = {0};

void v2_init(void) {
    tiku_persist_init(&store);
    /* Re-register: updates pointer,
       preserves existing data! */
    tiku_persist_register(&store,
        "cal", cal_fram, 16);

    uint8_t cal[16];
    tiku_mem_arch_size_t len;
    tiku_persist_read(&store,
        "cal", cal, 16, &len);
    /* cal = {0x42, 0x37} -- survived! */
    /* write_count = 1 -- preserved! */
}
```

How It Works

Example 4: MPU-Protected Persistent Writes

```c
/* Combine MPU scoped_write with persist_write for maximum safety */
typedef struct {
    tiku_persist_store_t *store;
    const uint8_t *data;
    tiku_mem_arch_size_t len;
} safe_write_ctx_t;

static void do_write(void *arg) {
    safe_write_ctx_t *ctx = (safe_write_ctx_t *)arg;
    tiku_persist_write(ctx->store, "cfg", ctx->data, ctx->len);
}

void safe_config_update(tiku_persist_store_t *store,
                        const uint8_t *new_cfg, tiku_mem_arch_size_t len)
{
    safe_write_ctx_t ctx = { store, new_cfg, len };
    tiku_mpu_scoped_write(do_write, &ctx);
    /* scoped_write guarantees:
       1. Interrupts disabled (no ISR can corrupt FRAM)
       2. NVM unlocked only during do_write()
       3. NVM re-locked automatically after return */
}
```

Best Practice

Example 5: Multi-Key Device Profile

```c
/* A sensor node with config, calibration, and error log */
static uint8_t __attribute__((section(".persistent")))
    cfg_fram[32]={0}, cal_fram[16]={0}, err_fram[64]={0};

void device_profile_init(tiku_persist_store_t *store) {
    tiku_persist_init(store);
    tiku_persist_register(store, "cfg", cfg_fram, sizeof(cfg_fram));
    tiku_persist_register(store, "cal", cal_fram, sizeof(cal_fram));
    tiku_persist_register(store, "err", err_fram, sizeof(err_fram));
}

void log_error(tiku_persist_store_t *store, uint8_t code) {
    uint8_t log[64];
    tiku_mem_arch_size_t len;
    /* Append to existing error log */
    tiku_persist_read(store, "err", log, sizeof(log), &len);
    if (len < sizeof(log)) {
        log[len] = code;
        tiku_persist_write(store, "err", log, len + 1);
    }
    /* Check if error log is wearing out FRAM */
    uint32_t writes;
    if (tiku_persist_wear_check(store, "err", &writes) == 1)
        rotate_error_log(store);   /* move to a fresh FRAM region */
}
```

API at a Glance

Function	Returns	Complexity	Description
`tiku_persist_init`	`err_t`	O(n)	Scan slots, recover valid entries
`tiku_persist_register`	`err_t`	O(n)	Map key to FRAM buffer
`tiku_persist_read`	`err_t`	O(n+k)	Copy FRAM→SRAM (k = value size)
`tiku_persist_write`	`err_t`	O(n+k)	Copy SRAM→FRAM, track wear
`tiku_persist_delete`	`err_t`	O(n)	Clear entry slot
`tiku_persist_wear_check`	`int`	O(n)	Check write count vs threshold

n = `TIKU_PERSIST_MAX_ENTRIES` (default 16), k = value length in bytes.

Error Handling

- NULL checks on every public function
- `ERR_NOT_FOUND` for missing keys
- `ERR_FULL` when store is full
- `ERR_NOMEM` for buffer/capacity overflows

Design Guarantees

- No dynamic allocation
- All FRAM buffers caller-provided
- Magic number validates entries
- Write count never wraps silently

Test Configuration

tests/tiku_test_config.h

```
/* Persistent store test flags
   (0 = off, 1 = on; enable as needed) */
#define TEST_PERSIST_INIT        0
#define TEST_PERSIST_REGISTER    0
#define TEST_PERSIST_WRITE_READ  0
#define TEST_PERSIST_SMALL_BUF   0
#define TEST_PERSIST_OVERFLOW    0
#define TEST_PERSIST_NOT_FOUND   0
#define TEST_PERSIST_DELETE      0
#define TEST_PERSIST_FULL        0
#define TEST_PERSIST_REBOOT      1
#define TEST_PERSIST_POWERCYCLE  0
#define TEST_PERSIST_WEAR        0
#define TEST_PERSIST_DUP_KEY     0
```

Host-native (no hardware):

```
gcc -std=c99 -Wall -Wextra -I. \
  tests/memory/test_mem_common.c \
  tests/memory/test_mem_arena.c \
  tests/memory/test_mem_persist.c \
  tests/memory/test_mem_mpu.c \
  kernel/memory/tiku_mem.c \
  kernel/memory/tiku_persist.c \
  kernel/memory/tiku_mpu.c \
  -o test_tiku_mem \
  && ./test_tiku_mem
```

Host mode provides portable stubs for arch NVM and MPU functions in `test_mem_common.c`.

Combined Test Binary

Arena, persistent store, and MPU tests share one test binary.

Test Cases Overview

#	Test Name	What It Verifies
10	Init zeroed	Zeroed store produces count = 0
11	Register & count	Count increments; NULL args rejected
12	Write/read	Round-trip: write data, read back, compare
13	Small buffer	ERR_NOMEM returned; out_len reports required size
14	Capacity overflow	Write exceeding capacity returns ERR_NOMEM
15	Not found	Read/write on missing key returns ERR_NOT_FOUND
16	Delete	Entry removed; count decremented; read fails
17	Store full	Fill all slots; one more returns ERR_FULL
18	Reboot survival	Re-init preserves entries with valid magic (software reset)
19	Power-cycle	Data survives full power removal and restore (hardware test)
20	Wear check	Returns 0 initially; 1 after threshold crossed
21	Duplicate key	Re-register updates pointer, preserves data

Test Deep Dive: Reboot Survival

Reboot survival — two-phase test across real hardware reset

```c
/* All variables in .persistent (FRAM) --- survive reset */
static uint8_t  __attribute__((section(".persistent")))
    phase = PHASE_WRITE;
static tiku_persist_store_t __attribute__((section(".persistent")))
    store = {0};
static uint8_t  __attribute__((section(".persistent")))
    fram_buf[16] = {0};

if (phase == PHASE_WRITE) {
    /* Phase 0: write data, then trigger software reset */
    mpu_state = tiku_mpu_unlock_nvm();
    tiku_persist_init(&store);
    tiku_persist_register(&store, "boot", fram_buf, 16);
    tiku_persist_write(&store, "boot", data, 3);
    phase = PHASE_VERIFY;
    tiku_mpu_lock_nvm(mpu_state);
    PMMCTL0 = PMMPW | PMMSWPOR;    /* Trigger software reset */
} else {
    /* Phase 1: verify data survived the reboot */
    tiku_persist_init(&store);
    TEST_ASSERT(store.count == 1);
    tiku_persist_read(&store, "boot", read_buf, sizeof(read_buf), &out_len);
    TEST_ASSERT(memcmp(read_buf, data, 3) == 0);  /* data intact */
}
```

Test Deep Dive: Power-Cycle Survival & Wear

Power-cycle survival (manual)

```c
/* Same two-phase pattern as reboot,
   but requires physical power removal
   instead of software reset */

if (phase == PHASE_WRITE) {
  mpu_state = tiku_mpu_unlock_nvm();
  tiku_persist_init(&store);
  tiku_persist_register(&store,
      "pwr", fram_buf, 16);
  tiku_persist_write(&store,
      "pwr", data, 5);
  phase = PHASE_VERIFY;
  tiku_mpu_lock_nvm(mpu_state);
  /* Blink LED and wait for user
     to physically remove power */
  for (;;) {
    tiku_common_led1_toggle();
    tiku_common_delay_ms(500);
  }
} else {
  /* After power restore: verify */
  tiku_persist_init(&store);
  TEST_ASSERT(store.count == 1);
```

Wear check test

```c
tiku_persist_store_t store;
uint8_t fram_buf[8];
uint32_t wc;

memset(&store, 0, sizeof(store));
tiku_persist_init(&store);
tiku_persist_register(&store,
    "wear", fram_buf, 8);

/* Initially below threshold */
int r = tiku_persist_wear_check(
    &store, "wear", &wc);
TEST_ASSERT(r == 0);
TEST_ASSERT(wc == 0);

/* Set above threshold */
store.entries[0].write_count =
    TIKU_PERSIST_WEAR_THRESHOLD + 1;

r = tiku_persist_wear_check(
    &store, "wear", &wc);
TEST_ASSERT(r == 1);
```

Summary

Persistent Store Design

- Key-value registry over FRAM
- Magic number validation
- Survives reboots and firmware updates
- Write-count wear monitoring
- All buffers statically declared
- NVM access routed through HAL
- Platform-independent kernel code

Test Coverage

- 12 dedicated test cases
- Runs on-target or host-native
- Covers: init, register, read/write, delete, full, reboot survival, power-cycle survival, wear, duplicates

Part of the Memory Module

The persistent store extends `tiku_mem.h` alongside the arena allocator — same header, same HAL, same test binary.

Thank You

TikuOS Persistent FRAM Key-Value Store

Non-Volatile Storage for Physical AI Endpoints

Repository:	`https://github.com/tikuos/tikuOS`
License:	Apache 2.0
API Header:	`kernel/memory/tiku_mem.h`
Tests:	`tests/memory/test_mem_persist.c`